Software Design Analysis Report

A. Project Details

Project title: Test Automation Framework with CI/CD Quality Gates for a European Investment Bank (case study)

Team / roll numbers: Abhay Pratap Singh (202401100100006); Aman Gupta (202401100100035); Amar Dwivedi(202401100100036); Abhishek Goswami (202401100100011)

Project objective: Replace manual regression testing, which took 2,000+ hours per release cycle on the bank's trading applications, with automated regression suites that run on every build and block deployment when tests fail. Based on the case study in Total Shift Left (2026).

Technology stack: Selenium, C#, CI/CD pipeline (the source does not name the CI/CD tool)

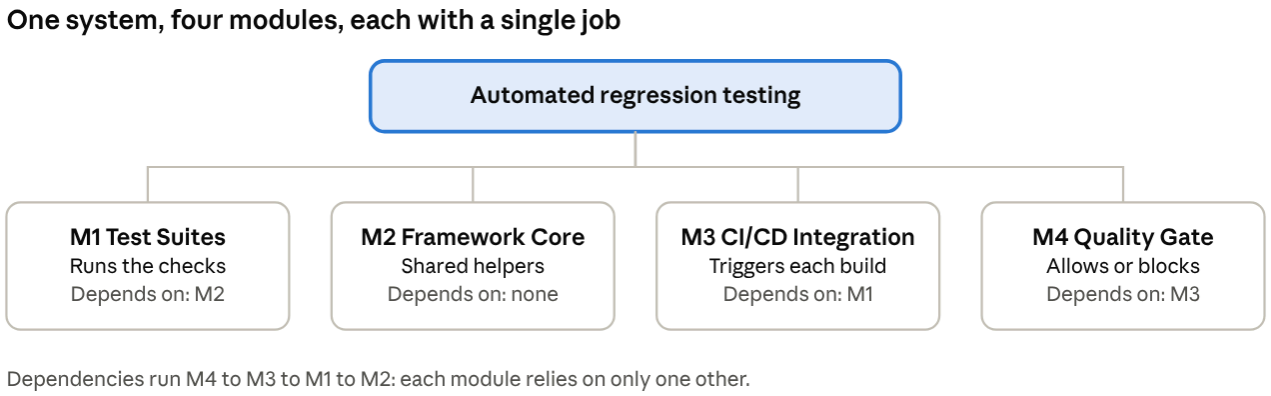
B. Module Identification and Modularization

Total number of modules: 4. The source describes the framework only at a high level, so this split is my analysis of it, not the bank's documented design. The system is divided by responsibility: writing and running checks, supporting those checks, triggering them automatically, and deciding whether a build may ship. Each part can change without rewriting the others.

Module ID	Module name	Main responsibility	Inputs / outputs	Depends on
M1	Regression Test Suites	Automate checks of existing trading-application features in the browser	In: build under test, test data. Out: pass/fail result per test	M2
M2	Test Framework Core	Provide shared Selenium and C# helpers such as browser control and waiting for page elements	In: commands from test suites. Out: browser actions, raw results	None

Module ID	Module name	Main responsibility	Inputs / outputs	Depends on
M3	CI/CD Integration	Start the regression suites on every build and collect their results	In: new build. Out: test run results	M1
M4	Quality Gate	Decide whether a build may be deployed and block deployment if any test fails	In: test run results. Out: allow or block decision	M3

Module hierarchy / decomposition diagram:



module hierarchy · 1 root, 4 modules

M3 and M4 sit on top of the test suites, and every module depends on at most one other.

C. Design Strategy

Selected strategy: ☐ Top-Down ☐ Bottom-Up ☒ Hybrid

Justification: The goal is clear from the top: no manual regression testing, and no failing build reaches deployment. That goal splits naturally into running checks, triggering them and gating releases, which is top-down. The checks themselves depend on shared Selenium helpers that must exist first and be reused by every test, which is bottom-up. Using only top-down risks duplicating helper code in each suite. Using only bottom-up risks building helpers nobody needs. The source does not state a design strategy, so this choice is my analysis.

Evidence:

- Whole system to modules (top-down): Automated release checks, then M1 Test Suites, M3 CI/CD Integration and M4 Quality Gate.

- Reusable components to integration (bottom-up): M2 Framework Core, then M1 Test Suites, then M3 CI/CD Integration, then M4 Quality Gate.

D. Design Approach

Selected approach: ☐ Function-Oriented ☒ Object-Oriented ☐ Mixed

Justification: Selenium test code in C# is commonly organised as classes: one class per application screen and test-case classes that use them. The main classes below are assumed for this analysis and are not stated in the source. Object-oriented design fits because many tests check the same screens, so each screen is described once and reused, and a change to a screen is made in one place.

- **TestCase** (M1): one automated check of a feature.
- **PageObject** (M1): hides the details of one application screen from the tests.
- **BrowserDriverManager** (M2): starts, controls and closes the browser.
- **TestRunner** (M3): runs all test cases for a build.
- **GateEvaluator** (M4): reads the results and returns allow or block.

Evidence: A function-oriented structure chart is not needed for an object-oriented design. The class list above and the module diagram show the structure.

E. Coupling Analysis

Three important module-to-module interactions. All three pass only simple data, so coupling stays low.

Module pair	Interaction / shared data	Coupling type / level	How to reduce dependency
M1 and M2	Test suites call framework helpers and pass commands, element names and test data as parameters	Data coupling (low)	Keep the helper interface small and stable; pass values as parameters, never shared variables
M3 and M1	The CI/CD step starts the suites and receives a pass/fail result per test	Data coupling (low)	Start suites through one standard test-runner command so M3 knows nothing about test internals
M4 and M3	The gate receives a summary of test results and returns allow or block	Data coupling (low)	Pass only a result summary; keep pass thresholds in configuration, not inside M3

F. Cohesion Analysis

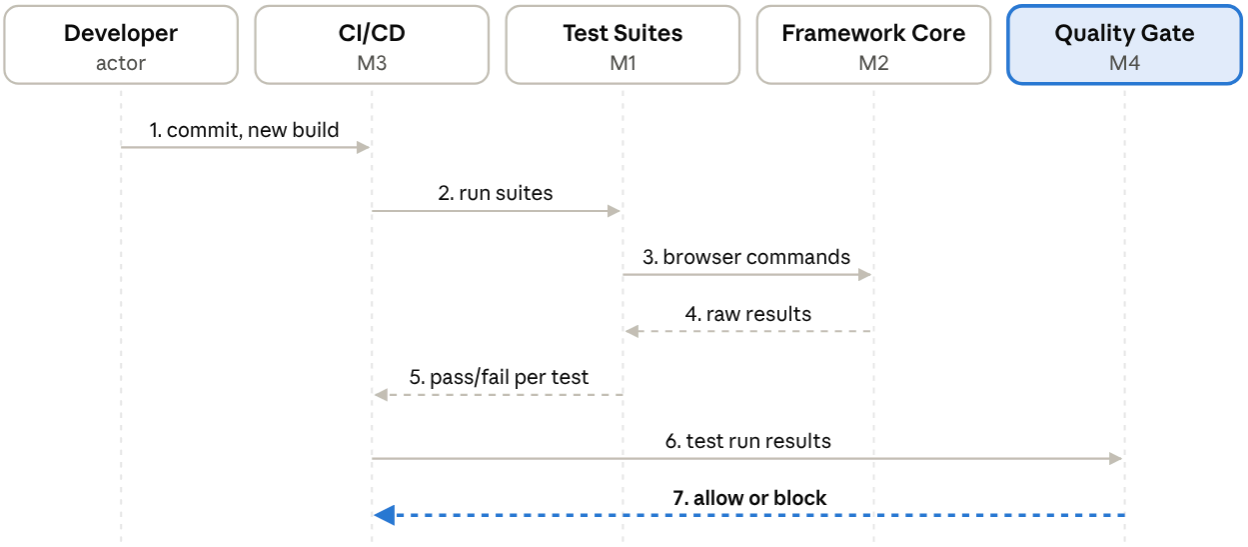
Three modules, each judged on how closely its internal tasks belong together.

Module	Main responsibility	Cohesion type / level	Improvement, if needed
M1 Regression Test Suites	Check that existing features still work	Functional (high)	Group tests by feature area so each group has one clear purpose
M2 Test Framework Core	Provide shared browser helpers	Logical (medium), because browser control and waiting helpers are different kinds of work	Split into browser control and utilities if the module grows
M4 Quality Gate	Decide allow or block from results	Functional (high)	None needed

G. Module Interaction and Design Quality

Sequence diagram of how the major modules exchange information for one build:

A build is deployed only after the quality gate allows it



If any test fails, the gate returns block and the build is not deployed.

sequence diagram · 5 participants, 7 messages

Design quality: each module passes only simple data to the next, so coupling is low, and each module has one main job, so cohesion is high. M2 is the weakest point and can be split if it grows.